Student Name:

Student Net Id:

Lab: Source Localization for EEG

EEG or Electroencephalography is a powerful tool for neuroscientists in understanding brain activity. In EEG, a patient wears a headset with electrodes that measures voltages at a number of points on the scalp. These voltages arise from ionic currents within the brain. A common *inverse problem* is to estimate the which parts of the brain caused the measured response. Source localization is useful in understanding which parts of the brain are involved in certain tasks. A key challenge in this inverse problem is that the number of unknowns (possible locations in the brain) is much larger than the number of measurements. In this lab, we will use LASSO regression on a real EEG dataset to overcome this problem and determine the brain region that is active under an auditory stimulus.

In addition to the concepts in the prostate LASSO demo you will learn to:

- Represent responses of multi-channel time-series data, such as EEG, using linear models
- Perform LASSO and Ridge regression
- Select the regularization level via cross-validation
- Visually compare the sparsity between the solutions

We first download standard packages.

```
import numpy as np
import matplotlib.pyplot as plt
import pickle

from sklearn.linear_model import Lasso, Ridge, ElasticNet
from sklearn.metrics import r2_score
from sklearn.model_selection import train_test_split
```

Load the Data

The data in this lab is taken from one of the sample datasets in the MNE website. The sample data is a recording from one subject who experienced some auditory stimulus on the left ear.

The raw data is very large (`1.5G`) and also requires that you install the `mne` python package. To make this lab easier, I have extracted and processed a small section of the data. The following command will download a `pickle` file `eeg_dat.p` to your local machine. If you do want to create the data yourself, the program to create the data is in this directory in the github repository.

```
fn_src ='https://drive.google.com/uc?export=download&id=1RzQpKONOcXSMxH2ZzOI4iVMiTgD6ttSl
fn_dst ='eeg_dat.p'

import os
from six.moves import urllib

if os.path.isfile(fn_dst):
    print('File %s is already downloaded' % fn_dst)
else:
    print('Fetching file %s [53MB].  This may take a minute..' % fn_dst)
    urllib.request.urlretrieve(fn_src, fn_dst)
    print('File %s downloaded' % fn_dst)
```

```
File eeg_dat.p is already downloaded
```

Now run the following command which will get the data from the `pickle` file.

```
import pickle
fn = 'eeg_dat.p'
with open(fn, 'rb') as fp:
    [X,Y] = pickle.load(fp)
```

To understand the data, there are three key variables:

- `nt` = number of time steps that we measure data
- `nchan` = number of channels (i.e. electrodes) measured in each time step
- `ncur` = number of currents in the brain that we want to estimate.

Each current comes from one brain region (called a *voxel*) in either the `x`, `y` or `z` direction. So,

```
nvoxels = ncur / 3
```

The components of the `X` and `Y` matrices are:

- `Y[i,k]` = electric field measurement on channel `i` at time `k`
- `X[i,j]` = sensitivity of channel `i` to current `j`.

Using `X.shape` and `Y.shape` compute and print `nt`, `nchan`, `ncur` and `nvoxels`.

```
# TODO 1
# nt = ...
# ncur = ...
# nchan = ...
# nvoxels

nt = X.shape[1]
ncur = X.shape[0]
nchan = Y.shape[0]
nvoxels = ncur // 3
print(f"nt = {nt}")
print(f"ncur = {ncur}")
```

```
print(f"nchan = {nchan}")
print(f"nvoxels = {nvoxels}")
```

```
nt = 22494
ncur = 305
nchan = 305
nvoxels = 101
```

Ridge Regression

Our goal is to estimate the currents in the brain from the measurements `Y`. One simple linear model is:

```
Y[i,k]  = \sum_j X[i,j]*W[j,k]+ b[k]
```

where `W[j,k]` is the value of current `j` at time `k` and `b[k]` is a bias. We can solve for the current matrix `W` via linear regression.

Howeever, there is a problem:

- There are `nt x ncur` unknowns in `W`
- There are only `nt x nchan` measurements in `Y`.

In this problem, we have:

```
number of measurements  << number of unknowns
```

We need to use regularization in these circumstances. We first try Ridge regression.

First split the data into training and test. Use the `train_test_split` function with `test_size=0.33`.

```
# TODO 2
# Xtr,Xts,Ytr,Yts = train_test_split(...)
Xtr,Xts,Ytr,Yts = train_test_split(X,Y,test_size=0.33)
```

Use the `Ridge` regression object in `sklearn` to fit the model on the training data. Use a regularization, `alpha=1`.

```
# TODO 3
# regr = Ridge(...)
regr = Ridge(alpha=1.0)
regr.fit(Xtr, Ytr)
```

```
▾ Ridge
Ridge()
```

Preict the values `Y` on both the training and test data. Use the `r2_score` method to measure the `R^2` value on both the training and test. You will see that `R^2` value is large for the training data, it is very low for the test data. This suggest that even with regularization, the model is over-fitting the data.

```
# TODO 4
# rsq_tr = ...
# rsq_ts = ...

Ytr_pred = regr.predict(Xtr)
Yts_pred = regr.predict(Xts)
rsq_tr = r2_score(Ytr, Ytr_pred)
rsq_ts = r2_score(Yts, Yts_pred)
print(f"R^2 on training data = {rsq_tr}")
print(f"R^2 on test data = {rsq_ts}")
```

```
R^2 on training data = 0.6033010670631425
R^2 on test data = 0.1713502447445955
```

Next, try to see if we can get a better `R^2` score using different values of `alpha`. Use cross-validation to measure the test `R^2` for 20 `alpha` values logarithmically spaced from `10^{-2}` to `10^{2}` (use `np.logspace()`). You can use regular cross-validation. You do not need to do `K`-fold.

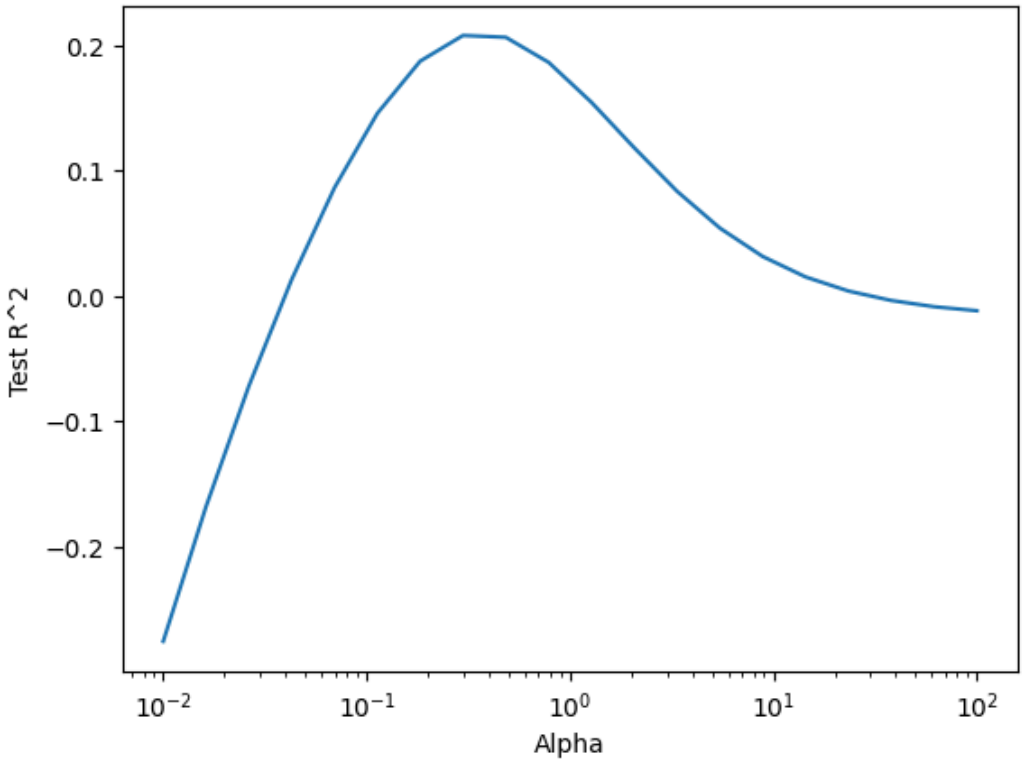
```
# TODO 5
alphas = np.logspace(-2, 2, 20)
rsq_ts_list = []

for a in alphas:
    regr_cv = Ridge(alpha=a)
    regr_cv.fit(Xtr, Ytr)
    Yts_pred_cv = regr_cv.predict(Xts)
    rsq_ts_list.append(r2_score(Yts, Yts_pred_cv))
```

Plot the test `R^2` vs. `alpha`. And print the maximum test `R^2`. You should see that the maximum test `R^2` is still not very high.

```
# TODO 6
plt.semilogx(alphas, rsq_ts_list)
plt.xlabel('Alpha')
plt.ylabel('Test R^2')
max_rsq = max(rsq_ts_list)
print(f"Maximum R^2 = {max_rsq}")
```

```
Maximum R^2 = 0.20770785952769635
```



Now, let's take a look at the solution.

- Find the optimal regularization `alpha` from the cross-validation
- Re-fit the model at the optimal `alpha`
- Get the current matrix `W` from the coefficients in the linear model. These are stored in `regr.coef_`. You may need a transpose
- For each current `j` compute `Wrms[j] = sqrt( sum_k W[j,k]**2 )` which is root mean squared current.

You will see that the vector `Wrms` is not sparse. This means that the solution that is found with Ridge regression finds currents in all locations.

```
# TODO 7
opt_alpha = alphas[np.argmax(rsq_ts_list)]

print(f"Optimal Alpha: {opt_alpha}")

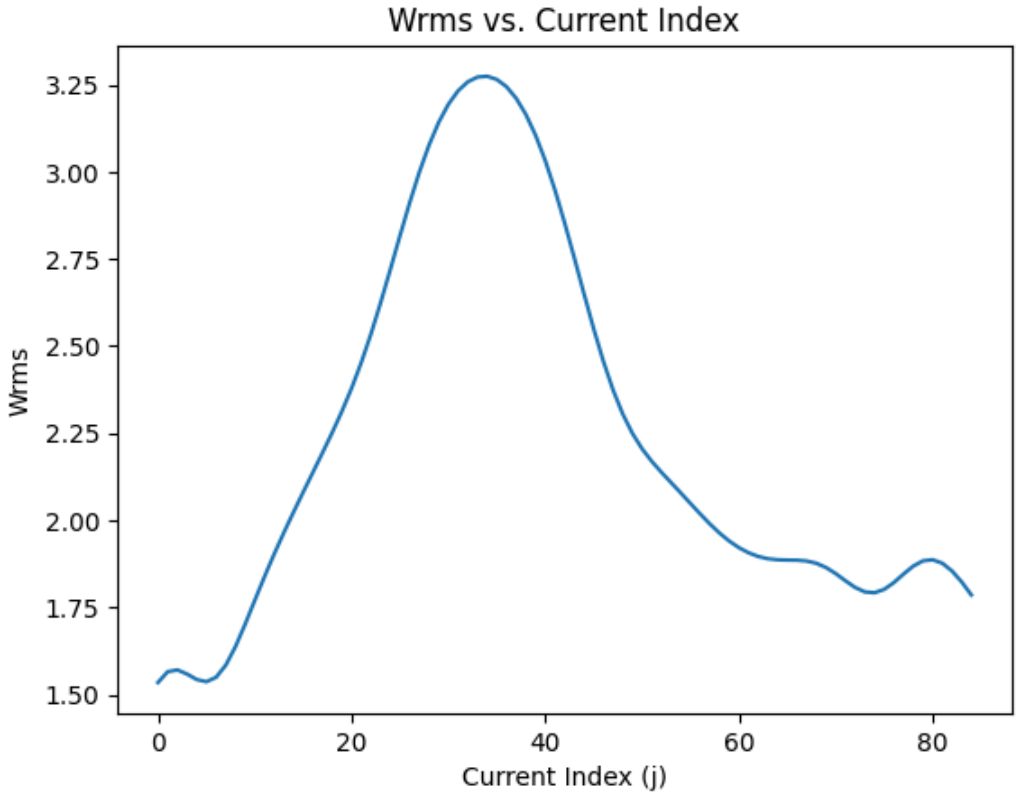
regr_opt = Ridge(alpha=opt_alpha)
regr_opt.fit(Xtr, Ytr)
W = regr_opt.coef_.T
Wrms = np.sqrt(np.sum(W**2, axis=0))

# print(f"Wrms = {Wrms}")

plt.plot(Wrms)
plt.xlabel('Current Index (j)')
plt.ylabel('Wrms')
plt.title('Wrms vs. Current Index')
```

```
# plt.grid(True)
plt.show()
```

```
Optimal Alpha: 0.29763514416313175
```



LASSO Regression

We can improve the estimate by imposing sparsity. Biologically, we know that only a limited number of brain regions should be involved in the reponse to a particular stimuli. As a result, we would expect that the current matrix `W[j,k]` to be zero for most values `j,k`. We can impose this constraint using LASSO regularization.

Re-fit the training data using the `Lasso` model with `alpha=1e-3`. Also set `max_iter=100` and `tol=0.01`. The LASSO solver is much slower, so this make take a minute.

```
# TODO 8
regr_lasso = Lasso(alpha=1e-3, max_iter=100, tol=0.01)
regr_lasso.fit(Xtr, Ytr)
```

```
Lasso
Lasso(alpha=0.001, max_iter=100, tol=0.01)
```

Now, test the model on the test data and measure the `R^2` value. You should get a much better fit than with the Ridge regression solution.

```
# TODO 9
Yts_pred_lasso = regr_lasso.predict(Xts)
rsq_ts_lasso = r2_score(Yts, Yts_pred_lasso)
print(f"R^2 on test data = {rsq_ts_lasso}")
```

```
R^2 on test data = 0.14401569291226532
```

We can now search for the optimal `alpha`. Use cross-validation to find the `alpha` logarithically space between `alpha=10^{-3}` and `alpha=10^{-4}`. Each fit takes some time, so use only 5 values of `alpha`. Also for each `alpha` store the current matrix. This way, you will not have to re-fit the model.

```
# TODO 10
alphas_lasso = np.logspace(-4, -3, 5)
rsq_ts_lasso_list = []
W_lasso_list = []

for a in alphas_lasso:
    regr_lasso_cv = Lasso(alpha=a, max_iter=100, tol=0.01)
    regr_lasso_cv.fit(Xtr, Ytr)

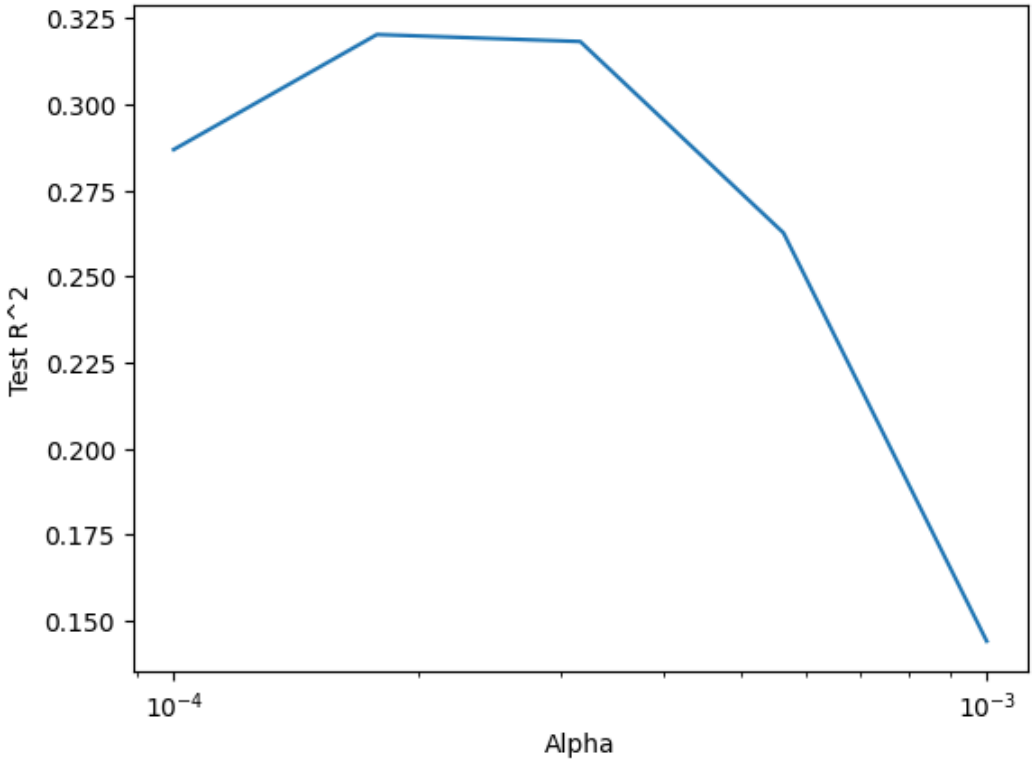
    Yts_pred_lasso_cv = regr_lasso_cv.predict(Xts)
    rsq_ts_lasso_list.append(r2_score(Yts, Yts_pred_lasso_cv))

    W_lasso_list.append(regr_lasso_cv.coef_.T)
```

Plot the `r^2` value vs. `alpha`. Print the optimal `r^2`. You should see it is much higher than with the best Ridge Regression case.

```
# TODO 11
plt.semilogx(alphas_lasso, rsq_ts_lasso_list)
plt.xlabel('Alpha')
plt.ylabel('Test R^2')
plt.show()

max_rsq_lasso = max(rsq_ts_lasso_list)
print(f"Maximum R^2 = {max_rsq_lasso}")
```

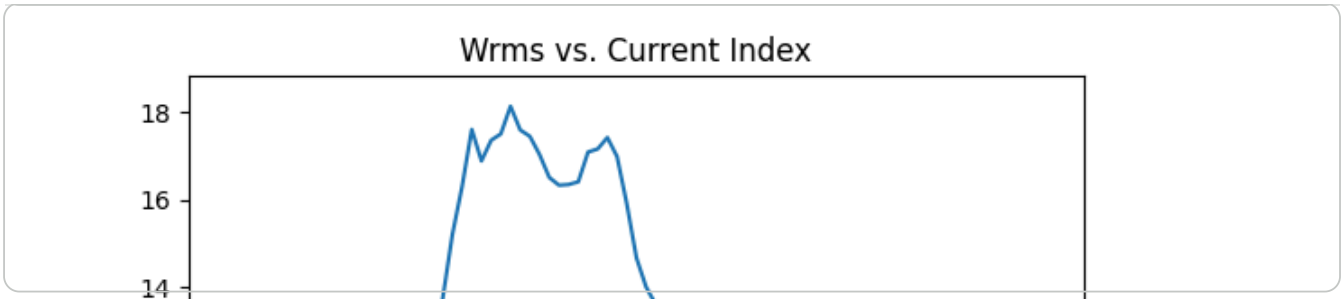



```
Maximum R^2 = 0.3203430826952208
```

Display the current matrix `W` for the optimal `alpha` as you did in the Ridge Regression case. You will see that is much sparser.

```
# TODO 12
opt_idx_lasso = np.argmax(rsq_ts_lasso_list)
W_opt_lasso = W_lasso_list[opt_idx_lasso]
Wrms_lasso = np.sqrt(np.sum(W_opt_lasso**2, axis=0))

plt.plot(Wrms_lasso)
plt.xlabel('Current Index (j)')
plt.ylabel('Wrms')
plt.title('Wrms vs. Current Index')
plt.show()
```



More fun (NOT GRADED)

If you want to more on this lab:

- Install the MNE python package. This is an amazing package with many tools for processing EEG data.
- In particular, you can use the above results to visualize where in the brain the currents sources are.
- You can also improve the fitting with more regularization. For example, we know that the